UNIVERSIDADE FEDERAL DE VIÇOSA
Campus de Rio Paranaíba
Sistemas de Informação
SIN 252 – Arquitetura de Computadores (T1 e T2) 2026
Prof. João Fernando Mari (joaof.mari@ufv.br)

Lista 01 – Conjunto de instruções

1) Considere que as variáveis a, b, c, d, e e f estão associadas aos registradores $s0 a $s5, respectivamente. Escreva os códigos em Assembly MIPS correspondentes às expressões:

a) `f = (a – b) + (c + d)`
b) `f = (a + b) – (c + d – e)`
c) `f = a + b + c + d + e`
d) `a = f * d`

2) Agora, considere que as variáveis a, b, c, d, e e f estão armazenadas sequencialmente na memória, a partir do endereço 32. Utilize lw e sw para mover dados entre memória e registradores. Assuma que cada variável ocupa uma palavra (4 bytes). Escreva os códigos MIPS para:

a) `f = (a – b) + (c + d)`
b) `f = (a + b) – (c + d – e)`
c) `a = a + 4`
d) `b = b – 10`

3) Considere dois vetores A e B de inteiros, cujos endereços base são 32 e 128, respectivamente. As variáveis f e h estão nos endereços 256 e 260. Escreva os códigos MIPS para:

a) `f = h + A[2]`
b) `A[16] = B[4] – f`
c) `B[8] = A[14] + B[12] + h`
d) `A[300] = h + A[300]`

4) Considere as variáveis f, g, h associadas aos registradores $s0, $s1, $s2, e o vetor V com base em $s3. Compile:

a) `f = (g + h) – V[4];`
b) `f = V[8] + g – h;`
c) `g = V[2] + V[3]`

5) As variáveis g, h estão em $s1, $s2, e o vetor V com base em $s0. Compile:

a) `V[10] = g – h + V[2];`
b) `V[0] = V[0] + 1;`
c) `V[2] = V[1] – V[2] + g`

6) Considere as variáveis a, b, c, d associadas a $s0–$s3 e vetor V em $s6. Converta os trechos em C para MIPS:

a)
```
if (a == b)
  a = b + c;
else
  a = b – c;
```

b)
```
if (a != b)
  a = b + c;
else
  a = b – c;
```
c)
```
if (a < b)
  a = b + c;
else
  a = b – c;
```
d)
```
if (V[8] <= a) {
  a = a + b + c;
  V[8] += a; }
else {
  a = a - b - c;
  V[8] += a;
}
```
e)
```
if(V[8] > b) {
  a = a + b + c;
  V[8] = V[8] + a;
} else {
  a = a – b – c;
  V[8] = V[8] + a;
}
```
f)
```
if(V[8] >= c) {
  a = a + b + c;
  V[8] = V[8] + a;
} else {
  a = a – b – c;
  V[8] = V[8] + a;
}
```

7) As variáveis a e b estão em $s0 e $s1, e o vetor V em $s6.

a)
```
while (b < 32) {
  a = a + a;
  V[4] = a + b;
  b = b + 1;
}
```
b)
```
while (V[0] <= a)
  V[0] = V[0] + b;
```

8) Implemente em MIPS o código das seguintes funções em C. Use os registradores adequados para passagem de parâmetros e retorno:

a)
```
int busca(int x, int n, int v[]) {
  int k = n-1;
  while (k >= 0 && v[k] != x)
    k--;
  return k;
}
```
b)
```
int busca_r(int x, int n, int v[]) {
  if (n == 0)
    return -1;
  if (x == v[n-1])
    return n-1;
```

```
    return busca_r(x, n-1, v);
  }
```

c)
```
  int soma_vetor(int v[], int n) {
    int soma = 0;
    for (int i = 0; i < n; i++)
      soma += v[i];
    return soma;
  }
```

9) Implemente o código MIPS para o seguinte procedimento folha:

```
int exemplo_folha(int g, int h, int i, int j) {
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

E para uma função não-folha que chama a função acima:

```
int exemplo_nao_folha(int a, int b) {
    int c;
    c = exemplo_folha(a, b, a, b);
    return c * 2;
}
```

Inclua comentários indicando:

- Empilhamento/desempilhamento de registradores
- Parâmetros nos registradores `$a0–$a3`
- Retorno em `$v0`
- Uso de `$ra` e `$sp`

10) Complete a tabela a seguir com base nas instruções do MIPS, indicando:

- Categoria: A – Aritmética, L – Lógica, T – Transferência, DC – Desvio Condicional, D – Desvio, P – Procedimento
- Formato: R, I, J
- Campos: op, rs, rt, rd, shamt, funct, endereço

E explique o funcionamento de cada instrução ao final, conforme exemplo:

- `add $s1, $s2, $s3`: Soma os valores de `$s2` e `$s3`, e armazena em `$s1`.

Instrução	Categoria	Formato	op	rs	rt	rd	shamt	funct	endereço
`add $s1, $s2, $s3`	A	R	0	$s2	$s3	$s1	0	32	n.a.
`sub $s1, $s2, $s3`									
`addi $s1, $s2, 10`									
`mul $s1, $s2, $s3`									
`lw $s1, 100($s2)`									
`sw $s1, 100($s2)`									
`and $s1, $s2, $s3`									
`or $s1, $s2, $s3`									
`nor $s1, $s2, $s3`									
`andi $s1, $s2, 10`									
`ori $s1, $s2, 10`									
`sll $s1, $s2, 10`									
`srl $s1, $s2, 10`									
`beq $s1, $s2, L`									
`bnq $s1, $s2, 10`									
`slt $s1, $s2, $s3`									
`slti $s1, $s2, 10`									
`j L`									
`jr $ra`									
`jal L`									

Observação: Caso algum aluno busque informações sobre Assembly MIPS-32 na Internet, é possível que se depare com **pseudoinstruções**. Essas instruções fazem parte da linguagem Assembly do MIPS, mas **não correspondem diretamente** a instruções da linguagem de máquina. Em vez disso, elas são convertidas pelo montador (assembler) em uma **sequência de instruções reais** do MIPS.

Como nosso objetivo é compreender a linguagem de montagem do MIPS para, posteriormente, entender e projetar uma CPU baseada nessa arquitetura, **não utilizaremos pseudoinstruções** neste curso.

Mais informações sobre pseudoinstruções podem ser encontradas em:
https://en.wikibooks.org/wiki/MIPS_Assembly/Pseudoinstructions